primamenthyl

This repository is organized as a staged pipeline:

1. extract per-sample features from indexed BAM files into DuckDB
2. run per-sample dbt models on each DuckDB file
3. merge all per-sample DuckDB files into one combined DuckDB file
4. run merged-file dbt models for cross-sample summaries
5. generate per-sample plots from the merged DuckDB file
6. train a logistic regression model on merged sample-level features

Pipeline Logic

1. `bam_processing/`: BAM/BAI -> per-sample DuckDB

`bam_processing` is the raw feature extraction layer.

Input: - one `.bai` file per sample - the matching BAM file - a reference FASTA

Output: - one DuckDB file per sample in `$OUTPUT_DIR/duckdb/<sample>.features.duckdb`

The extractor writes four base tables:

- `quant__records` One row per retained alignment record. Includes: `align_id`, `query_id`, `is_read1`, `template_length`, `reference`, `position`, `start_position`, `end_position`.
- `quant__methylation` One row per methylation event, keyed by `align_id`, with a read-local `position`.
- `quant__motif_counts` Aggregated end-motif counts.
- `quant__samples` Sample metadata such as `sample`, `total_records`, `total_fragments`, `age`, and `group_name`.

This layer is the only part that reads BAM data directly. Everything downstream works from DuckDB.

2. `dbt_models/`: per-sample transformations

The dbt project has two logical groups of models:

- `tag:individual` Models that run on each per-sample DuckDB file during `orchestration/02_run_dbt_models.sh`.
- `tag:merged` Models that run on the merged DuckDB file during `orchestration/04_run_merged_dbt_models.sh`.

Per-sample models add derived structure on top of the raw `quant__*` tables. Examples:

- `stg_record_methylation_counts` Counts methylation events per `align_id`.
- `stg_record_fragments` Assigns a `fragment_id` to records sharing the same `start_position` and `end_position`.

- `stg_fragment_pairing` Counts records per fragment and marks whether a fragment is paired.
- `stg_methylation_positions` Converts methylation offsets into genomic positions by joining `quant__methylation` with `quant__records`.

These models stay inside each sample's DuckDB file.

3. `dbt_models/merge_duckdb_files.py`: per-sample DuckDB -> merged DuckDB

After per-sample extraction and modeling, all sample DuckDB files are merged into:

- `output/duckdb/all_samples.features.duckdb`
- by default this resolves to `$OUTPUT_DIR/duckdb/all_samples.features.duckdb`

The merge script unions tables and views across samples. For relations that do not already have a `sample` column, it prepends one so the merged file keeps sample identity.

This merged database is the handoff point for cohort-level summaries, plotting, and modeling.

4. `dbt_models/`: merged-file distributions and summaries

Merged dbt models build cross-sample summary tables directly in the merged DuckDB file. Current examples include:

- `fragment_length_distribution`
- `start_position_distribution`
- `end_position_distribution`
- `end_motif_distribution`
- `methylation_position_distribution`

These tables are designed for downstream plotting and modeling.

Example:

- `methylation_position_distribution` joins merged `quant__records` and `quant__methylation`, resolves genomic methylation positions, bins them into 100000 bp windows, and groups by `sample` and `reference`.

5. `plotting/`: merged DuckDB -> PDFs

The plotting scripts are standalone Python programs that:

- read the merged DuckDB file
- query precomputed distribution tables
- write one PDF per sample into `output/plots`

Current plot families:

- fragment length distribution
- start position distribution
- end position distribution
- methylation position distribution
- 5' end motif distribution
- 3' end motif distribution

These scripts use `matplotlib.figure.Figure` and `seaborn` directly, without `plt`.

6. `models/`: merged DuckDB -> classifier outputs

The `models` directory contains the logistic regression training script.

Feature set:

- `age`
- all methylation-position bins across chromosome 21 from `methylation_position_distribution`

Target:

- `group_name` reduced to `ctrl` vs `als`

Outputs in `output/models`:

- `logistic_regression_metrics.csv` with precision, sensitivity, and F1 score
- `logistic_regression_roc_curve.pdf`

If the merged data does not contain both classes, the script does not crash. It writes a skipped status and an explanatory ROC PDF instead.

Directory Roles

- bam_processing Raw BAM feature extraction.
- dbt_models Per-sample and merged DuckDB transformations.
- plotting Standalone plotting scripts using the merged DuckDB file.
- models Standalone ML training script using merged derived features.
- orchestration Shell scripts that run the pipeline in order.

End-to-End Run

From the repo root:

```
export BAM_DIR=/path/to/bam_indexes
export FASTA_PATH=/path/to/reference.fa
export JAX_PLATFORMS=cpu
export OUTPUT_DIR=output

bash orchestration/00_run_all.sh
```

Or step by step:

```
bash orchestration/01_run_bam_processing.sh
bash orchestration/02_run_dbt_models.sh
bash orchestration/03_merge_duckdb_files.sh
bash orchestration/04_run_merged_dbt_models.sh
bash orchestration/05_run_plotting.sh
bash orchestration/06_run_model.sh
```

Notes

- `FASTA_PATH` is required for `orchestration/01_run_bam_processing.sh` because `bam_processing` needs the reference FASTA to derive motif and methylation-position context.
- `OUTPUT_DIR` is the base output directory. The orchestration layer writes DuckDB files under `"$OUTPUT_DIR/duckdb"`, plots under `output/plots`, and model outputs under `output/models`.
- `JAX_PLATFORMS` controls the JAX backend used by `bam_processing`. In practice this should usually be `cpu`, or `METAL` on Apple Silicon when `jax-metal` is installed in the `bam_processing` environment.
- Per-sample dbt models run before merging.
- Merged dbt models run after merging.
- Plotting and modeling both depend on the merged DuckDB file and the merged dbt summary tables.
- The root pipeline is DuckDB-centered: BAM is only touched once, at the extraction step.